2. 8_PUZZEL USING A*

```
import heapq
GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def manhattan(state):
    distance = 0

    for i, tile in enumerate(state):
        if tile != 0:
            goal = tile - 1
            distance += abs(i // 3 - goal // 3)
            distance += abs(i % 3 - goal % 3)

    return distance

def get_successors(state):
    blank = state.index(0)
    row, col = blank // 3, blank % 3

    moves = [
        (-1, 0, "UP"),
        (1, 0, "DOWN"),
        (0, -1, "LEFT"),
        (0, 1, "RIGHT")
    ]

    for dr, dc, action in moves:
```

```
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_blank = nr * 3 + nc
            new_state = list(state)
            new_state[blank], new_state[new_blank] = (
                new_state[new_blank], new_state[blank]
            )

            yield tuple(new_state), action

def solve(start):
    frontier = [(manhattan(start), 0, start)]
    parent = {start: (None, None)}
    cost = {start: 0}

    while frontier:
        f, g, state = heapq.heappop(frontier)

        if g != cost[state]:
            continue

        if state == GOAL:
            path = []
            current = state

            while parent[current][0] is not None:
                previous, action = parent[current]
                path.append((action, current))
```

```
            current = previous

        path.reverse()
        return path

    for next_state, action in get_successors(state):
        new_cost = g + 1

        if new_cost < cost.get(next_state, float("inf")):
            cost[next_state] = new_cost
            parent[next_state] = (state, action)

            heapq.heappush(
                frontier,
                (
                    new_cost + manhattan(next_state),
                    new_cost,
                    next_state
                )
            )

    return None

def print_board(state):
    for i in range(0, 9, 3):
        print(" ".join(
            "_" if tile == 0 else str(tile)
            for tile in state[i:i + 3]
        ))
```

```
def main():
    print("8-Puzzle A* Search")
    print("Enter the initial state (use _ for blank):")

    start = []

    try:
        for i in range(3):
            row = input(f"Enter row {i + 1}: ").split()

            if len(row) != 3:
                print("Solution Not Found")
                print("Solution Cost = 0")
                return

            start.extend(0 if x == "_" else int(x) for x in row)

    except (ValueError, EOFError):
        print("Solution Not Found")
        print("Solution Cost = 0")
        return

    start = tuple(start)

    if len(start) != 9 or sorted(start) != list(range(9)):
        print("Solution Not Found")
        print("Solution Cost = 0")
        return
```

```
inversions = sum(
    1
    for i in range(9)
    for j in range(i + 1, 9)
    if start[i] != 0
    and start[j] != 0
    and start[i] > start[j]
)

if inversions % 2 != 0:
    print("Solution Not Found")
    print("Solution Cost = 0")
    return

path = solve(start)

if path is None:
    print("Solution Not Found")
    print("Solution Cost = 0")
    return

if not path:
    print("Goal State Reached")
    print("Solution Cost = 0")
    return

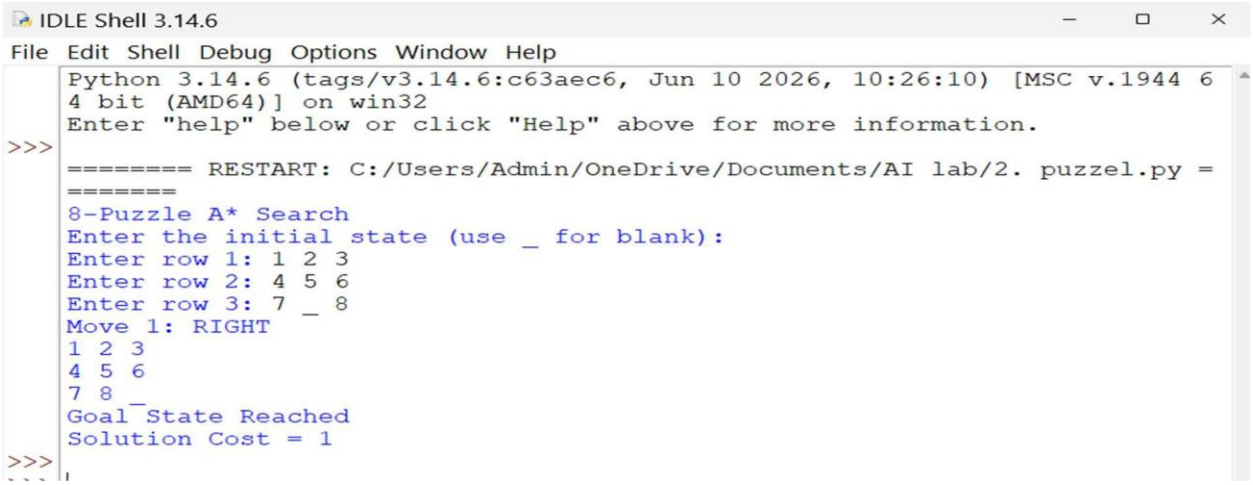
for i, (action, state) in enumerate(path, 1):
    print(f"Move {i}: {action}")
    print_board(state)
```

```
    print("Goal State Reached")
    print(f"Solution Cost = {len(path)}")

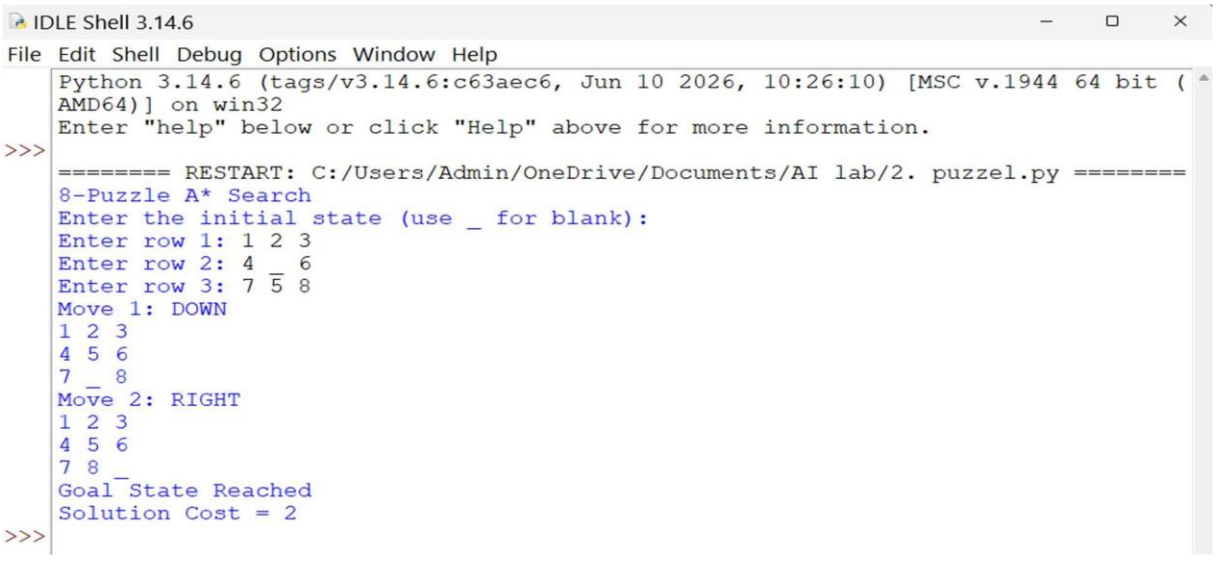
if __name___== "__main__":
    main()
```

Output:

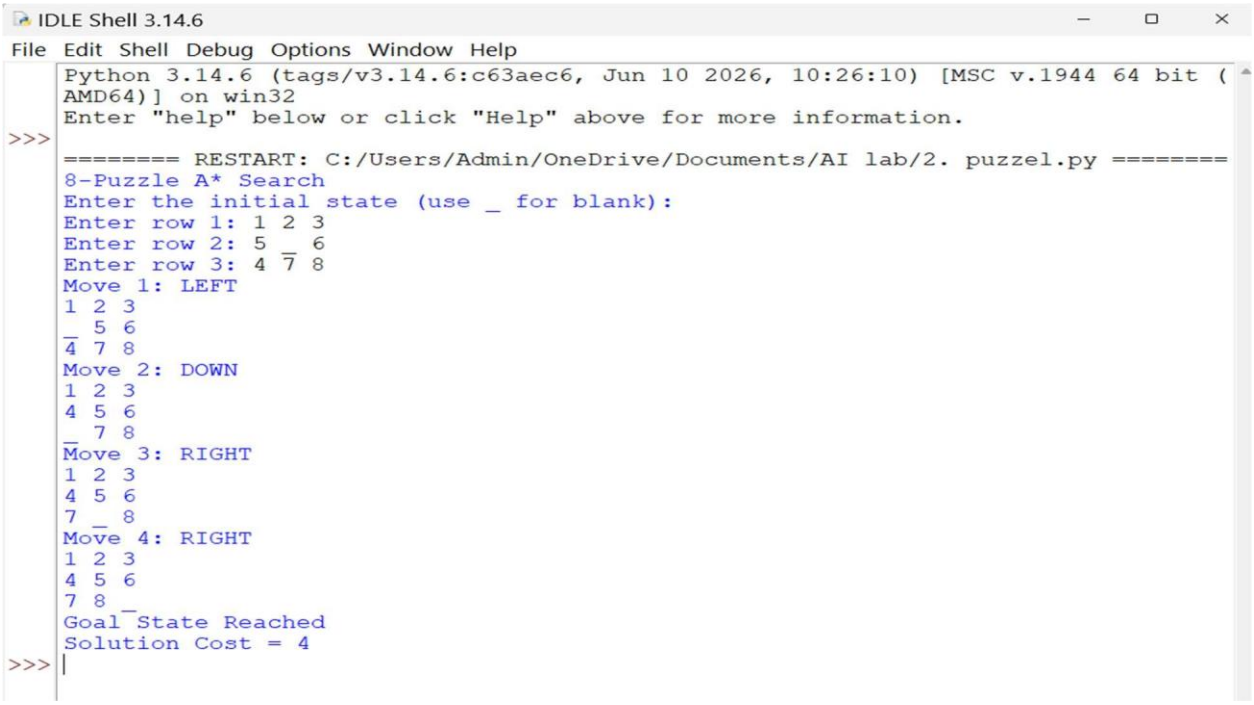
Test case 1: One – Move Solution



Test case 2: Two-Move Solution



Test case 3: Move Complex Solution



Test case 3: solution not found

